AuthWho

Continuous Behavioural Biometric Authentication Engine for Web Sessions

> **ONE-LINE PITCH**
>
> AuthWho keeps checking who is behind a session after login, using how the person types, moves the mouse and interacts with the page. When confidence drops it reacts in proportion (quiet monitoring, step-up verification, re-authentication or session termination), and when there is not enough behaviour to judge it says so explicitly instead of guessing.

1. Executive proposal

AuthWho is a continuous authentication system for web applications. It is designed for settings where a hijacked session is expensive: web applications in general, banking and fintech portals, administrator consoles, online examination portals and remote-work tools. Rather than fully trusting a session once the password and multi-factor authentication (MFA, a second check such as a one-time code or phone approval) have been passed, AuthWho learns a behavioural profile for each user from keystroke timing, mouse motion and page-interaction patterns, and compares live behaviour against that profile for the whole session. The output is a continuous trust score with an attached confidence value, which a policy engine converts into proportionate actions.

The defining principle is that **trust is continuous, earned from behaviour, and every decision must be explainable and privacy-preserving**. The system must be able to distinguish "this is the enrolled user" from "this is probably someone else" and from "there is not enough signal to tell". This makes the project a strong demonstration of behavioural biometrics, streaming machine learning, anomaly detection, adversarial machine learning, privacy engineering and secure system design. It needs no special hardware: all signals come from an ordinary laptop's keyboard, mouse or touchpad through standard browser events, and the whole system uses free, open-source software.

2. AuthWho in simple words

Today, most websites check who you are only once, when you log in. After you type your password and approve the code on your phone, the website trusts everything that happens next until you log out. If someone else takes over at that point, for example by stealing your logged-in session through a fake website, or simply by sitting down at your unlocked laptop, the website has no way of noticing that it is no longer you.

AuthWho fixes this by paying attention to *how* you use the computer, not *what* you type. Everyone has a personal typing rhythm, a personal way of moving and clicking the mouse, and personal habits when filling in forms or moving around a website. AuthWho learns this "behavioural signature" and quietly keeps comparing it with what is happening now. If the behaviour stops looking like you, it first watches more carefully, then asks for a quick extra check before anything risky (such as sending money), and ends the session if the check fails. It never records the letters you type, only timing and movement, and if you are simply reading or away from the keyboard, it honestly says "I cannot tell yet" instead of making a guess.

> **IN SIMPLE WORDS**
>
> Think of a bank cashier who has served the same customer for years. One day a stranger walks in with that customer's genuine passbook. The passbook is real, but the cashier notices that "something is off" in the way this person behaves. The passbook is your password and OTP. The cashier's familiarity with how you behave is AuthWho.

3. Problem statement

In everyday words: websites check your identity once at the door and then never look again, and attackers exploit exactly that gap.

Authentication in most systems is a point-in-time event. Once login and MFA succeed, the server issues a

session token (a random value, usually stored in a session cookie, that proves “this browser already logged in”), and every later request carrying that token is trusted. NIST’s guidelines already treat reauthentication and session management as part of authentication [13], and zero trust architecture (a model in which no request is trusted by default) calls for continuous evaluation of trust [14]. In practice, these gaps remain:

- MFA is a one-time check: after it succeeds, the rest of the session inherits full trust regardless of who is actually operating it.
- Stolen session cookies and tokens can be replayed from another machine. Adversary-in-the-middle phishing kits (fake login pages that relay traffic to the real site) capture the cookie after the victim completes MFA, so MFA is bypassed rather than broken.
- Unattended, unlocked computers let anyone nearby act inside an authenticated session.
- Credential sharing and insider misuse: the person using an account is not always its owner.
- Bots and remote-access malware (software that lets an attacker control a victim’s computer) can act inside a real session from the victim’s own device and network.
- Account takeover after credential stuffing (automated login attempts using passwords leaked from other sites) produces sessions that look valid from the server’s point of view.
- IP address, geolocation and device-fingerprint rules help, but an attacker on the same device, or behind a proxy in the same city, passes all of them.

4. Proposed solution

In everyday words: a small piece of code in the web page measures the rhythm of typing and mouse movement, a server compares it with the user’s usual rhythm, and a set of rules decides what to do.

- A lightweight browser JavaScript SDK (software development kit, a small library added to the web application) that listens to standard keyboard, mouse, scroll and focus events and captures only timing and motion metadata, never the characters typed.
- Keystroke dynamics features (measurements of typing rhythm): dwell time (how long a key is held down), flight time (the gap between releasing one key and pressing the next), digraph and trigraph latencies (timings across pairs and triples of keys), key rollover (pressing the next key before releasing the previous one) and the rhythm of errors and corrections [1, 2, 3].
- Mouse dynamics features (measurements of pointer movement): velocity, acceleration, curvature, pause distribution, click timing and movement efficiency (straight-line distance divided by path length) [5, 6, 7].
- Interaction features: navigation patterns, hesitation before sensitive actions, form-filling order, scrolling behaviour and use of copy and paste.
- Per-user profile enrollment from a consented initial period of normal use, stored as an encrypted template (a compact numerical summary of behaviour, not raw events).
- Continuous trust scoring over sliding windows of recent actions, fusing channels in the spirit of the penalty-and-reward trust model used for combined keystroke and mouse continuous authentication [9].
- A policy engine that maps the trust score, its confidence and the sensitivity of the requested action to an outcome.
- Bot and replay detection (machine-like timing, impossible motion, repeated behaviour) and drift-aware profile updates (behaviour changes slowly over time) that accept data only from verified sessions.
- An administrator view that explains every decision in terms of which channels and features moved the score.

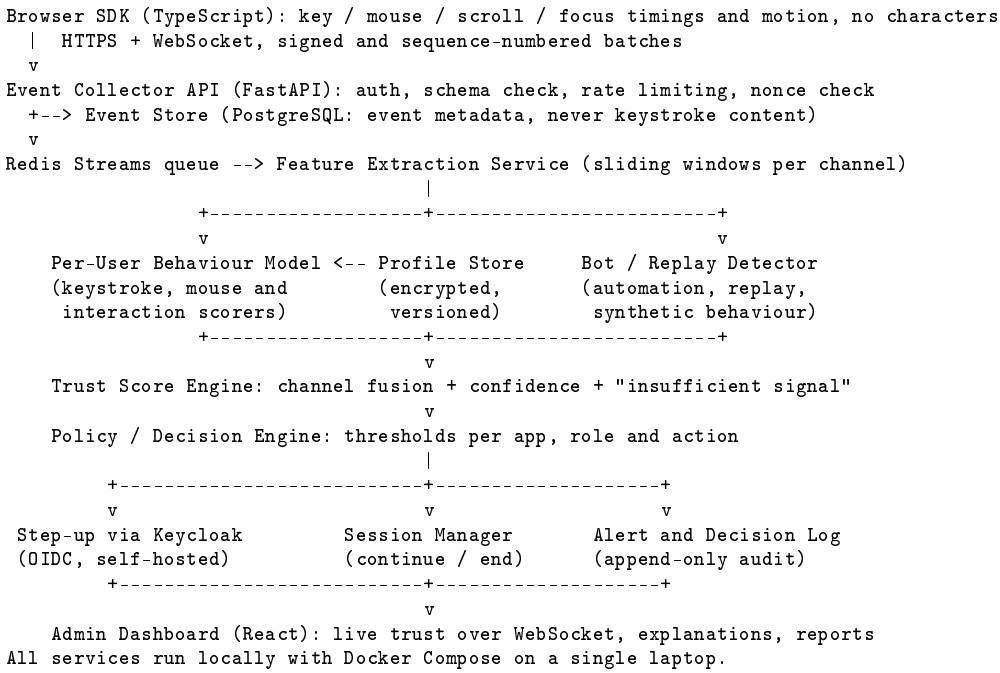
5. Concrete example

A finance analyst logs in with a password and MFA and then steps away from the desk. Either a phishing kit has already captured the session cookie and an attacker replays it on another machine, or a colleague uses the unlocked laptop directly. Within the first few dozen keystrokes, the new operator’s dwell and flight times sit well outside the analyst’s usual range, and the mouse moves in faster, straighter strokes with different pauses before clicks. AuthWho’s trust score falls steadily over the next few seconds while its confidence rises, because both channels agree. The operator then tries to add a new payee and export a customer report, two actions marked as high risk. Before either completes, AuthWho requires step-up verification (an extra identity check at that moment); when that fails, the action is blocked, the session is ended and an incident record lists

the features and time windows behind the decision. Earlier, while the real analyst was only scrolling through a report, AuthWho did not guess: it reported “insufficient behavioural signal” and left the session under normal rules.

6. Industry-grade architecture

In everyday words: signals flow from the browser to a server that turns them into measurements, scores them against the user’s profile, and hands the result to a rule engine that decides what to do.



> **ARCHITECTURE PRINCIPLE**
>
> Separate raw signal capture, scoring and policy so that each can change independently; never capture or store keystroke content; keep scoring asynchronous and off the user’s request path so that the application never waits for the model; persist every decision together with the evidence and confidence behind it rather than only the final verdict.

7. The trust model

In everyday words: AuthWho never simply says “yes” or “no”; it says how sure it is, why, and what it is allowed to do at that level of certainty.

Every decision is a trust statement backed by signals. “Typing rhythm over the last 60 keystrokes matches the enrolled profile” is a direct observation. “The current operator is probably not the enrolled user” is an inference supported by several channels diverging together. “The operator is the enrolled user” is recorded as “Cannot determine” if the user has been idle and there are too few recent events to score.

Trust level	Example signal pattern	Allowed system action
High confidence	Keystroke and mouse channels both match the profile within normal variation	Continue silently
Moderate deviation	One channel deviates (for example a new keyboard, tiredness, or an unusual task) while others match	Keep monitoring; apply stricter thresholds for sensitive actions
Strong deviation	Several channels diverge together, or timing looks bot-like or replayed	Step-up authentication before continuing
Critical	Strong deviation combined with a high-risk action, or confirmed automation	Block the action and end the session
Insufficient signal	User idle, only scrolling, or too few events in the window	Report “Cannot determine”; fall back to normal session rules

8. Continuous authentication pipeline

In everyday words: these are the steps AuthWho repeats every few seconds, from noticing a key press to deciding whether the session is still safe.

1. Signal capture: the SDK records key-down and key-up times with a coarse key-category code (letter, digit, space, backspace, modifier, navigation), pointer coordinates with high-resolution timestamps, clicks, scrolls and focus changes.
2. Secure transport: events are batched, sequence-numbered and signed with a per-session key, then sent over HTTPS or WebSocket so that tampered, reordered or replayed batches are rejected.
3. Windowing: the stream is cut into overlapping windows defined by a number of actions (for example 50 keystrokes or 30 mouse strokes) rather than a fixed time, so idle periods do not dilute the evidence.
4. Feature extraction: each window becomes a feature vector: dwell, flight and digraph statistics for keystrokes [2]; velocity, acceleration, curvature and pause statistics per mouse stroke [5, 6]; and navigation and hesitation features for interaction.
5. Per-user scoring: each channel is scored against the user's own model (one-class models for small enrollment sets, learned embeddings for larger ones) [2, 4, 7], producing a match score and a confidence.
6. Fusion into a trust score: channel scores are combined with weights that depend on how much evidence each channel had, and the running trust value is updated with a penalty-and-reward rule so that single odd actions do not end a session while sustained deviation does [9].
7. Bot, replay and adversarial checks: timing regularity, physically implausible motion, near-exact matches to previously stored windows and population-statistics patterns are flagged independently of the user model [11, 12].
8. Policy decision: the policy engine combines trust, confidence and action sensitivity into continue, monitor, step-up or terminate, and writes the decision with its evidence to the audit log.
9. Feedback and profile update: the profile is updated only from windows in sessions that passed step-up verification or were otherwise confirmed genuine, with limits on how far a single update may move it, to prevent profile poisoning.

9. Explainability, privacy & adversarial robustness

In everyday words: AuthWho must be able to say why it acted, must not collect anything it does not need, and must be hard to fool on purpose.

The model never acts on its own. It produces scores and confidences; a separate, human-readable policy decides what happens, and every decision records which channels and features drove the score, the size of each deviation, the event window and the model version. A channel with too little data is marked as missing rather than estimated, making explainability a system property rather than a dashboard feature.

- No plaintext keystrokes are ever captured: the SDK sends only timings and key-category codes, so passwords, messages and form contents never leave the page.
- Feature extraction runs in the browser where practical; profile templates are encrypted at rest, versioned and fully deletable; users see a plain-language notice and give consent at enrollment.
- The design follows the data-minimisation and purpose-limitation principles of India's Digital Personal Data Protection Act, 2023 [15] and the EU GDPR [16], which explicitly lists behavioural characteristics within its definition of biometric data.
- Mimicry defence: Tey et al. showed that people given feedback about a victim's typing can learn to imitate it closely enough to defeat several keystroke verifiers [11], so AuthWho fuses independent channels, uses features that are hard to observe from outside (rollover, correction rhythm, mouse curvature) and never shows its internal score to the operator.
- Synthetic-behaviour defence: Serwadda and Phoha showed that keystroke timings generated from population statistics can substantially increase verifier error rates [12], so AuthWho checks whether a window looks like the population average rather than like the user.
- Replay and automation defence: signed, sequence-numbered batches block network replay; similarity checks against stored windows catch recorded behaviour played back; timing regularity and missing micro-corrections flag script- or ML-generated input.
- Slow profile poisoning (gradually training the profile towards an attacker) is limited by verified-only updates,

capped update size and alerts on sustained drift.

10. Product orientation & UX

In everyday words: this is what administrators and users actually see and click on.

- Session list and trust timeline: live trust scores and confidence per session, charted over time with decision points marked.
- Channel breakdown: separate keystroke, mouse and interaction contributions for any moment in the session.
- Plain-language explanations: each decision rendered as a sentence, for example “typing rhythm and mouse speed both diverged from this user’s profile for 40 seconds”.
- Policy editor (thresholds per application, role and action) and a guided enrollment and re-enrollment flow for new users or new hardware.
- “What we collect” page: an end-user privacy page listing exactly which signals are captured and which are not.
- Exportable incident report: timeline, decisions, contributing features and confidence for a flagged session.
- Live demo mode: an evaluator uses the author’s logged-in session on a test application and watches the trust score fall in real time until step-up is triggered.

11. Engineering quality bar

In everyday words: the project is built the way real production systems are built, but sized for one student laptop.

- Event-driven scoring so that capture, feature extraction, scoring and policy are independently testable and replaceable.
- FastAPI for the collector and query APIs, and WebSockets for live session state on the dashboard.
- PostgreSQL for users, policies, sessions and decisions; Redis Streams for the event queue and Redis keys for hot session state.
- Model versioning with a local MLflow server, so every decision traces to the exact model that made it; all services in one Docker Compose file.
- Unit, integration and replay-based tests with CI on the GitHub Actions free tier; observability with self-hosted Prometheus and Grafana: scoring latency, queue lag, dropped batches, step-up rate and model errors.
- An append-only, hash-chained audit log of trust decisions so that past decisions cannot be silently altered.
- Secrets kept in environment files excluded from version control.

12. Tech stack and zero-cost setup

In everyday words: every tool listed here is free, and all of it runs on an ordinary laptop.

Layer	Technology	Purpose / justification	Cost
Client capture	TypeScript browser SDK	Reads standard DOM keyboard, mouse, scroll and focus events; no plugins or extra hardware	Free, open source
Backend	Python, FastAPI	Collector and query APIs, WebSocket push, fast to develop and test	Free, open source
Queue	Redis Streams	Ordered, replayable event stream between collector and feature service	Free, open source
Machine learning	NumPy, pandas, scikit-learn, PyTorch	One-Class SVM, Gaussian Mixture Models and autoencoders as small-data baselines (following the detector comparison in [2]), plus Isolation Forest as a standard extra baseline; Siamese or triplet LSTM encoders following TypeNet [4]; 1D CNN or Transformer encoders for mouse strokes, building on [7]	Free, open source
Model serving	ONNX Runtime	Fast CPU inference of exported models inside the scoring service	Free, open source

Layer	Technology	Purpose / justification	Cost
Experiment tracking	MLflow (local)	Model versions, parameters and metrics for each experiment	Free, open source
Storage	PostgreSQL, Redis	Durable records of users, policies and decisions; hot session state	Free, open source
Step-up identity	Keycloak (self-hosted, OIDC)	Re-authentication and step-up flows through a standard identity protocol	Free, open source
Dashboard	React, TypeScript, Recharts or Chart.js	Session list, trust timelines, channel breakdowns and policy editor	Free, open source
Infrastructure	Docker Compose	Whole stack on one laptop; optional free-tier hosting for a public demo	Free
CI	GitHub Actions	Tests and container builds on every push	Free tier
Monitoring	Prometheus, Grafana	Metrics and dashboards for latency, lag and error rates	Free, open source
Datasets	CMU, Aalto, Balabit, SapiMouse, consented volunteer data	Benchmarking and training (see Section 13)	Free

The complete system runs on a single student laptop with a standard keyboard and mouse or touchpad, inside an ordinary web browser. It needs no cameras, fingerprint readers, IoT devices or other sensors, and no paid services, SIEMs (security information and event management platforms) or ML platforms. This is a strength as well as a constraint: AuthWho works on the hardware users already have.

13. Evaluation plan

In everyday words: these are the numbers that show whether AuthWho catches impostors without annoying genuine users.

Metric	What it measures
False Acceptance Rate (FAR)	How often an impostor is wrongly accepted as the genuine user
False Rejection Rate (FRR)	How often the genuine user is wrongly flagged as an impostor
Equal Error Rate (EER)	The point where FAR equals FRR; a single number for comparing models, lower is better
ROC-AUC	How well genuine and impostor behaviour are separated across all thresholds (1.0 is perfect)
Time-to-detect an impostor	How many seconds, or how many actions, an impostor can perform before step-up or termination, in the spirit of the impostor-actions measure used in [9]
Bot / replay detection rate	Share of scripted, synthetic and replayed sessions that are caught
Scoring latency (p50 / p95)	Typical and worst-case time from an event arriving to an updated trust score
Robustness under drift	Error rates when the genuine user switches keyboard, is tired, or uses the system at a different time of day
Robustness under attack	Error rates under mimicry [11] and population-statistics synthetic attacks [12]
User friction	Number of step-up prompts shown to a genuine user per day

Benchmarks will use free public datasets: the CMU Keystroke Dynamics benchmark dataset (51 subjects typing the same password 400 times) [2], the Aalto 136 million keystrokes dataset (free-text typing from about 168,000 volunteers) [3], the Balabit Mouse Dynamics Challenge dataset [8] and the SapiMouse dataset [7], plus a small consented dataset from classmates collected through the AuthWho SDK to test live multi-channel fusion. As targets, the keystroke channel should match or beat the best EER of about 9.6% reported on the CMU benchmark [2]; deep free-text models such as TypeNet report a few per cent with large training sets [4]. For the fused keystroke and mouse trust score on the consented dataset, the aim is an EER below 5% within a window of about one minute of active use, with a genuine-user step-up rate of no more than one prompt per day.

14. Phased roadmap

In everyday words: the project is built in eight steps, and each step has a clear test of whether it worked.

Phase	Deliverable	Success criterion
P0	Data collection SDK, consented dataset and event schema	Reproducible capture with no character content, verified by inspection

Phase	Deliverable	Success criterion
P1	Keystroke features and baseline models on the CMU dataset	EER comparable to published baselines on the standard protocol
P2	Mouse and interaction features on Balabit and SapiMouse	Per-user mouse models with measured EER and AUC
P3	Multi-channel fusion and continuous trust score	Fused score outperforms each single channel on consented data
P4	Bot, replay and adversarial defences	Scripted, replayed and synthetic sessions detected at a measured rate
P5	Policy engine and Keycloak step-up	End-to-end step-up and termination triggered by trust thresholds
P6	Admin dashboard, explanations and live demo mode	Every decision explained; live takeover demo works reliably
P7	Tests, containers, monitoring and deployment	CI green, one-command Docker Compose start-up, dashboards live

15. Research papers referred

In everyday words: these are the published studies that AuthWho builds on, and what each one contributes.

#	Paper (authors, year, venue)	Key idea	How AuthWho uses it
[1]	Monrose and Rubin, 2000, Future Generation Computer Systems	Keystroke latencies and durations can identify and authenticate users, including during free-text typing	Foundation for using timing only, and for monitoring typing throughout a session rather than at login
[2]	Killourhy and Maxion, 2009, IEEE/IFIP DSN	Compared 14 anomaly detectors on a common keystroke benchmark; source of the CMU dataset	Baseline one-class detectors, evaluation protocol and EER target for the keystroke channel
[3]	Dhakal, Feit, Kristensson and Oulasvirta, 2018, ACM CHI	Large-scale analysis of typing behaviour from 136 million keystrokes	Realistic feature ranges, rollover as a feature, population baseline for synthetic-attack checks
[4]	Acien, Morales, Monaco, Vera-Rodriguez and Fierrez, 2022, IEEE T-BIOM	TypeNet: LSTM-based Siamese and triplet networks for free-text keystroke authentication at scale	Embedding model for users with enough data; comparison with one-class baselines
[5]	Ahmed and Traore, 2007, IEEE TDSC	Mouse dynamics as a behavioural biometric using speed, direction and action-type features	Core mouse feature set and the observation that longer observation lowers error
[6]	Shen, Cai, Guan, Du and Maxion, 2013, IEEE TIFS	Mouse authentication from frequent operation patterns; analyses behavioural variability	Procedural mouse features and drift-aware thresholds
[7]	Antal, Fejér and Buza, 2021, IEEE SACI	SapiMouse dataset and deep feature learning (1D CNN) on short raw mouse segments	Mouse-stroke encoder and benchmark for short-window mouse scoring
[8]	Balabit Mouse Dynamics Challenge, 2016, GitHub	Public mouse dataset with genuine and impostor test sessions	Benchmark for mouse-channel impostor detection
[9]	Mondal and Bours, 2017, Neurocomputing	Continuous authentication combining keystroke and mouse, with a trust model and impostor-action metrics	Trust update rule, channel fusion and time-to-detect metric
[10]	Frank, Biedert, Ma, Martinovic and Song, 2013, IEEE TIFS	Touchalytics: touch strokes as a behavioural biometric for continuous authentication	Stroke-level features for touchpad input and the case for continuous auth as an extra layer, not a replacement
[11]	Tey, Gupta and Gao, 2013, NDSS	Humans trained with feedback can imitate another user's keystroke timing	Mimicry threat model, hidden scores and multi-channel fusion as defences
[12]	Serwadda and Phoha, 2013, ACM TISSEC	Population-statistics forgeries raise error rates of keystroke verifiers	Synthetic-behaviour threat model and population-likeness check
[13]	NIST SP 800-63B, Digital Identity Guidelines	Authenticator assurance, reauthentication and session management requirements	Framing of step-up and reauthentication as part of session security
[14]	NIST SP 800-207, Zero Trust Architecture, 2020	Trust must be evaluated continuously per request, not granted once	Architectural motivation for a continuous trust score and policy engine

Research gap. Most of the work above studies a single channel (keystroke or mouse) offline on fixed datasets. AuthWho combines several channels, scores them continuously inside a live web session, treats "insufficient signal" as an explicit outcome rather than forcing a decision, defends against the mimicry, synthetic and replay

attacks documented in the literature, and does all of this with zero special hardware on free software.

Glossary

Term	Plain-English meaning
MFA	Multi-factor authentication: proving identity with more than one thing, e.g. password plus phone code
Session cookie	A value stored by the browser that tells the website "this browser is already logged in"
Step-up authentication	Asking for an extra identity check in the middle of a session, just before a risky action
Keystroke dynamics	Recognising a person by the rhythm and timing of their typing
Dwell time	How long a single key is held down
Flight time	The gap between releasing one key and pressing the next
Mouse dynamics	Recognising a person by how they move and click the mouse
Trust score	AuthWho's running estimate of how likely it is that the enrolled user is operating the session
FAR	False Acceptance Rate: how often an impostor is wrongly let through
FRR	False Rejection Rate: how often the real user is wrongly challenged
EER	Equal Error Rate: the point where FAR and FRR are equal; lower is better
Drift	Slow, natural change in a person's behaviour over time
Mimicry attack	An attacker deliberately copying the victim's typing or mouse style
Replay attack	Re-sending previously recorded data or behaviour to pretend to be the user
OIDC	OpenID Connect: a standard way for an app to ask an identity provider to log a user in or re-verify them

References

[1] F. Monrose and A. D. Rubin, "Keystroke dynamics as a biometric for authentication," *Future Generation Computer Systems*, vol. 16, no. 4, pp. 351–359, 2000.

[2] K. S. Killourhy and R. A. Maxion, "Comparing anomaly-detection algorithms for keystroke dynamics," in *Proc. IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2009, pp. 125–134. Dataset: https://www.cs.cmu.edu/~keystroke/

[3] V. Dhakal, A. M. Feit, P. O. Kristensson and A. Oulasvirta, "Observations on typing from 136 million keystrokes," in *Proc. ACM CHI Conference on Human Factors in Computing Systems*, 2018.

[4] A. Acien, A. Morales, J. V. Monaco, R. Vera-Rodriguez and J. Fierrez, "TypeNet: Deep learning keystroke biometrics," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 4, no. 1, 2022.

[5] A. A. E. Ahmed and I. Traore, "A new biometric technology based on mouse dynamics," *IEEE Transactions on Dependable and Secure Computing*, vol. 4, no. 3, pp. 165–179, 2007.

[6] C. Shen, Z. Cai, X. Guan, Y. Du and R. A. Maxion, "User authentication through mouse dynamics," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 1, pp. 16–30, 2013.

[7] M. Antal, N. Fejér and K. Buza, "SapiMouse: Mouse dynamics-based user authentication using deep feature learning," in *Proc. IEEE International Symposium on Applied Computational Intelligence and Informatics (SACI)*, 2021.

[8] Balabit, "Mouse Dynamics Challenge," dataset, 2016. https://github.com/balabit/Mouse-Dynamics-Challenge

[9] S. Mondal and P. Bours, "A study on continuous authentication using a combination of keystroke and mouse biometrics," *Neurocomputing*, vol. 230, pp. 1–22, 2017.

[10] M. Frank, R. Biedert, E. Ma, I. Martinovic and D. Song, "Touchalytics: On the applicability of touchscreen input as a behavioral biometric for continuous authentication," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 1, pp. 136–148, 2013.

[11] C. M. Tey, P. Gupta and D. Gao, "I can be you: Questioning the use of keystroke dynamics as biometrics," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2013.

[12] A. Serwadda and V. V. Phoha, "Examining a large keystroke biometrics dataset for statistical-attack openings," *ACM Transactions on Information and System Security*, vol. 16, no. 2, 2013.

[13] P. A. Grassi et al., "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017. https://doi.org/10.6028/NIST.SP.800-63b

[14] S. Rose, O. Borchert, S. Mitchell and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, 2020. https://doi.org/10.6028/NIST.SP.800-207

[15] Government of India, *The Digital Personal Data Protection Act, 2023*.

[16] European Parliament and Council, *Regulation (EU) 2016/679 (General Data Protection Regulation)*, Arts. 4(14), 5(1)(c).

References are provided for the mentor; AuthWho is an independent personal student project, not affiliated with any cited author or organisation.